



Kubernetes im Einsatz neu: Kubernetes Day 2

helm, Monitoring, Fehlersuche und Troubleshooting



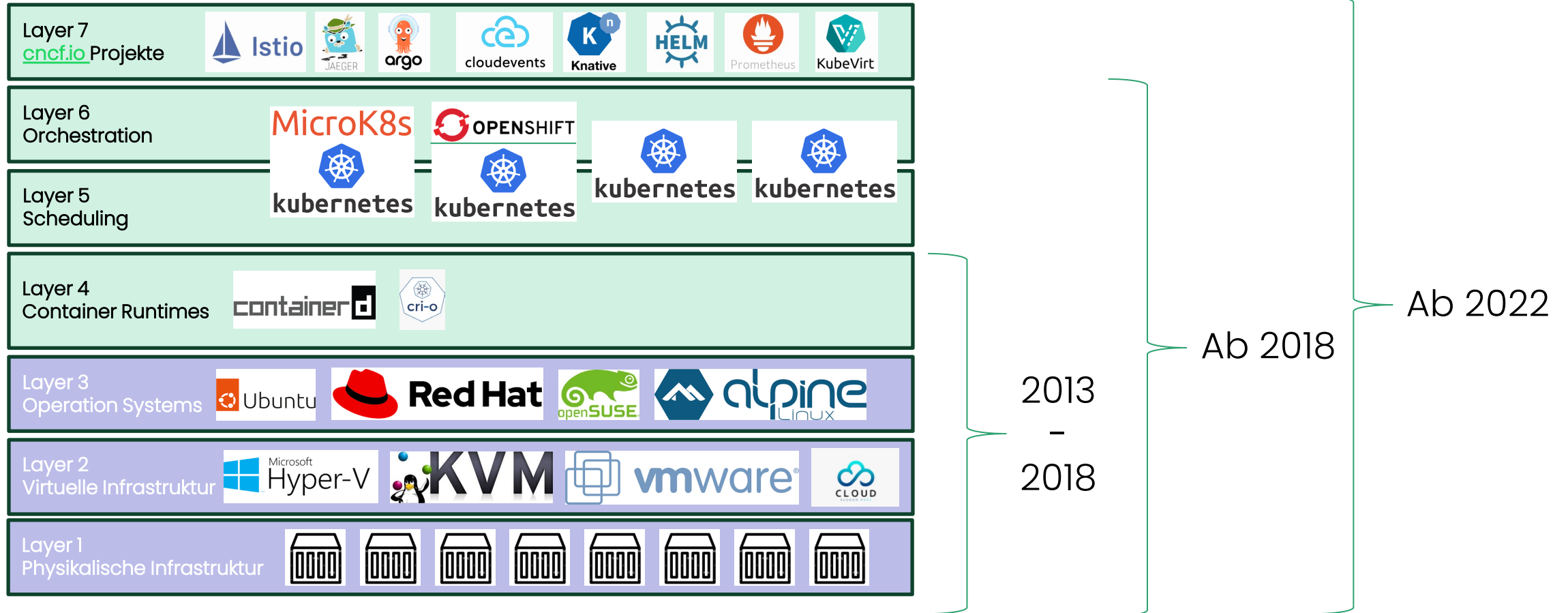
Lernziele

- Sie haben einen Überblick über Erweiterte Funktionen im Kubernetes Cluster.

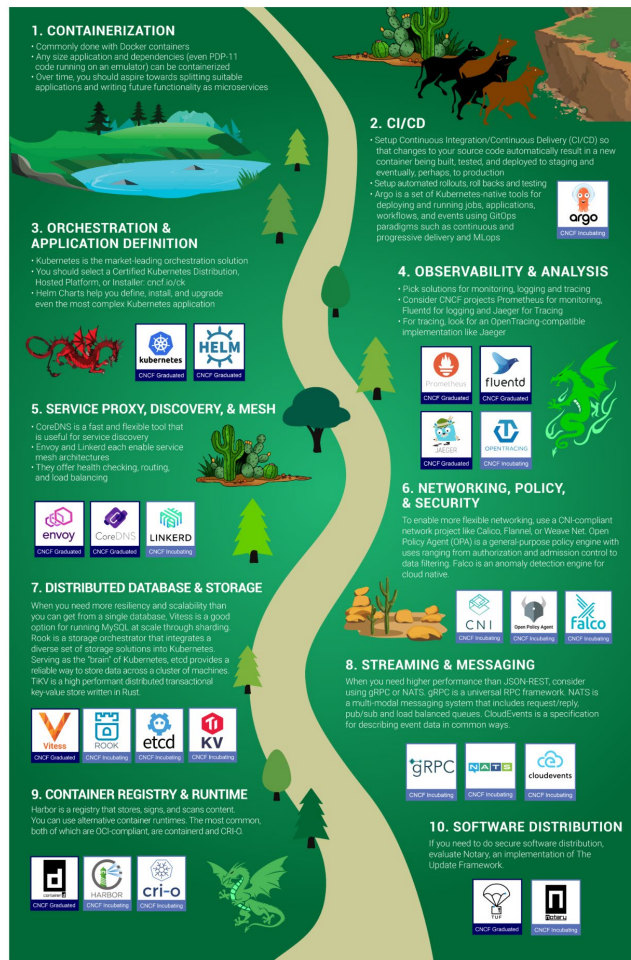
Zeitlicher Ablauf

- Helm – der Paketmanager für Kubernetes
- Operator Pattern
- Monitoring
- Fehlersuche und Troubleshooting
- Security (Empfehlungen)
- Reflexion
- Lernzielkontrolle
- Bonus: Continuous Integration (CI) und Delivery (CD) und Static Application Security Testing (SAST)

Layer 5 - 7 = Cloud-native oder auch Day 2



Cloud Native Trail Map (dt.: Wanderkarte)



- Die Cloud-Native-Trail-Karte bietet einen Überblick für Unternehmen, die ihre Cloud-native (Docker/Kubernetes) Reise starten.

- Containerization (Docker)
- CI/CD (dt.: Fortlaufende Integration, Verteilung)
- Orchestration (Kubernetes, Helm)
- Observability & Analysis (Logging)
- Service Mesh and Discovery
- Networking, Policy, Security
- Messaging (Serverless)
- ...

Quelle: <https://github.com/cncf/landscape/blob/32731f3986a654915fca1a0b1c3cadb6a867d818/README.md>

Cloud-native: Oekosystem





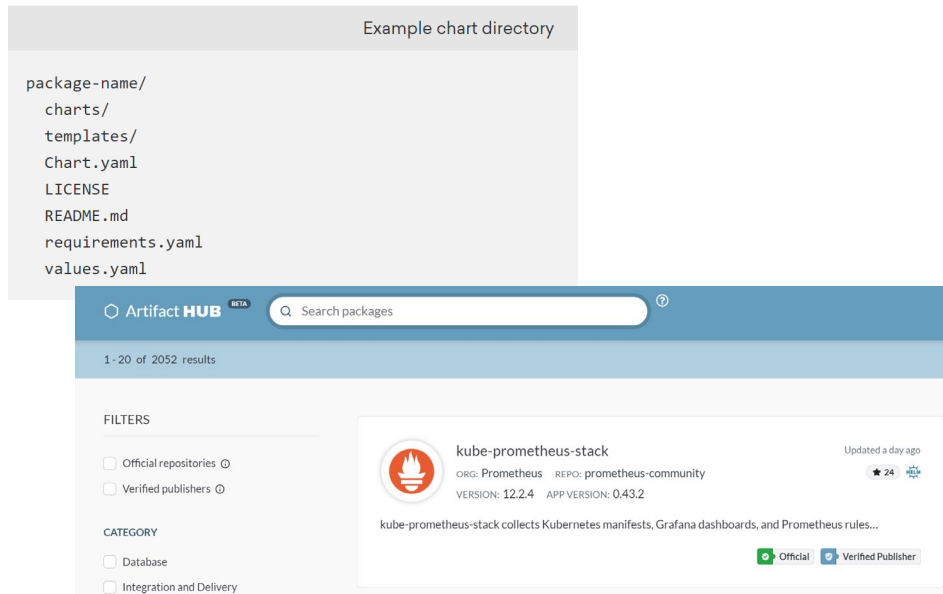
Helm

Der Paketmanager für Kubernetes

Helm

- Fast jede Programmiersprache und jedes Betriebssystem verfügt über einen eigenen Paketmanager, der bei der Installation und Wartung von Software hilft.
- Helm bietet die gleichen grundlegenden Funktionen wie viele der Paketmanager, mit denen Sie möglicherweise bereits vertraut sind, z. B. Debian apt oder Python pip.
- Helm kann:
 - ◇ Software installieren.
 - ◇ Software-Abhängigkeiten automatisch auflösen und installieren.
 - ◇ Software aktualisieren.
 - ◇ Konfigurieren von Software (Paketen) in Repositories
 - ◇ Abrufen von Softwarepaketen aus Repositories

Helm – die drei grossen Konzepte



- Ein **Chart** ist ein Helmpaket. Es enthält alle Ressourcendefinitionen, die zum Ausführen einer Anwendung, eines Tools oder eines Dienstes in einem Kubernetes-Cluster erforderlich sind.
 - Ein **Repository** (neu Artefakt-Hub) ist der Ort, an dem Diagramme gesammelt und freigegeben werden können.
 - Ein **Release** ist eine Instanz eines **Charts**, das in einem Kubernetes-Cluster ausgeführt wird. Ein **Chart** kann mehrmals im selben Cluster installiert werden.
- `helm repo add stable https://<repo>`
 - `helm install db1 stable/mysql`
 - `helm install db2 stable/mysql`



Helm Charts

Example chart directory

```
package-name/  
charts/  
templates/  
Chart.yaml  
LICENSE  
README.md  
requirements.yaml  
values.yaml
```

Chart.yaml

```
1  apiVersion: v1  
2  appVersion: "3.9"  
3  description: Deploy a basic Alpine Linux pod  
4  home: https://helm.sh/helm  
5  name: alpine  
6  sources:  
7  - https://github.com/helm/helm  
8  version: 0.1.0
```

values.yaml

```
1  Name: my-alpine
```

Beispiele: <https://github.com/helm/charts>

templates/alpine-pod.yaml

```
1  apiVersion: v1  
2  kind: Pod  
3  metadata:  
4    name: "{{.Release.Name}}-{{.Values.Name}}"  
5    labels:  
6      # The "app.kubernetes.io/managed-by" label is used to track which tool  
7      # deployed a given chart. It is useful for admins who want to see what  
8      # releases a particular tool is responsible for.  
9      app.kubernetes.io/managed-by: "{{.Release.Service | quote }}"  
10     # The "app.kubernetes.io/instance" convention makes it easy to tie a release  
11     # to all of the Kubernetes resources that were created as part of that  
12     # release.  
13     app.kubernetes.io/instance: "{{.Release.Name | quote }}"  
14     app.kubernetes.io/version: "{{.Chart.AppVersion }}"  
15     # This makes it easy to audit chart usage.  
16     helm.sh/chart: "{{.Chart.Name}}-{{.Chart.Version}}"  
17     values: "{{.Values.Name}}"  
18  spec:  
19     # This shows how to use a simple value. This will look for a passed-in value  
20     # called restartPolicy. If it is not found, it will use the default value.  
21     # {{default "Never" .restartPolicy}} is a slightly optimized version of the  
22     # more conventional syntax: {{.restartPolicy | default "Never"}}  
23     restartPolicy: {{default "Never" .Values.restartPolicy}}  
24     containers:  
25     - name: waiter  
26       image: "alpine:{{.Chart.AppVersion }}"  
27       command: ["/bin/sleep","9000"]
```

Helm Repositories (dann Artefakt-Hub)

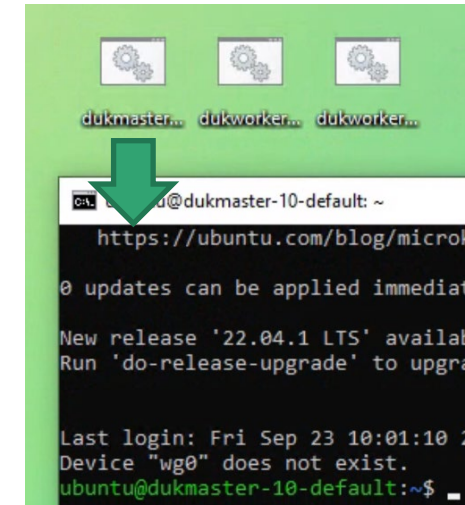
- Artifact Hub ist eine webbasierte Anwendung, mit der Pakete und Konfigurationen für CNCF-Projekte gesucht, installiert und veröffentlicht werden können.
- Dies können beispielsweise Helmdiagramme, Falco-Konfigurationen, OPA-Richtlinien (Open Policy Agent) und OLM-Operatoren (Operator Lifecycle Manager) sein.
- **Neu:** jede Container Registry kann Helm Charts speichern. Beispiel GitLab
 - ◇ `git clone https://gitlab.com/ch-mc-b/autoshop-ms/infra/helm.git`
 - ◇ `cd helm`
 - ◇ `helm package autoshop`
 - ◇ `helm push autoshop-<version>.tgz oci://registry.gitlab.com/ch-mc-b/autoshop-ms/infra/helm`
- **Resultat:** https://gitlab.com/ch-mc-b/autoshop-ms/infra/helm/container_registry/



Übung: Installation Wordpress



- Installiert das Blog System [Wordpress](#).
- Wechselt dazu, via Desktop Icon, in die "dukmaster" VM
- Führt die Helm Installation, wie im [Artifact](#) beschrieben aus:
 - ◇ `helm repo add bitnami https://charts.bitnami.com/bitnami`
 - ◇ `helm install my-wp bitnami/wordpress`
- Schaut auch die erstellten Ressourcen im Dashboard an.
 - ◇ <https://dukmaster-10-default.mshome.net:30443/>
- Welche Ressourcen wurden alle erzeugt und warum?
- Tipp: `--dry-run` erzeugt die Ressourcen nicht, sondern zeigt nur die YAML-Dateien an
 - ◇ `helm install --dry-run my-wpdry bitnami/wordpress`





Operator Pattern

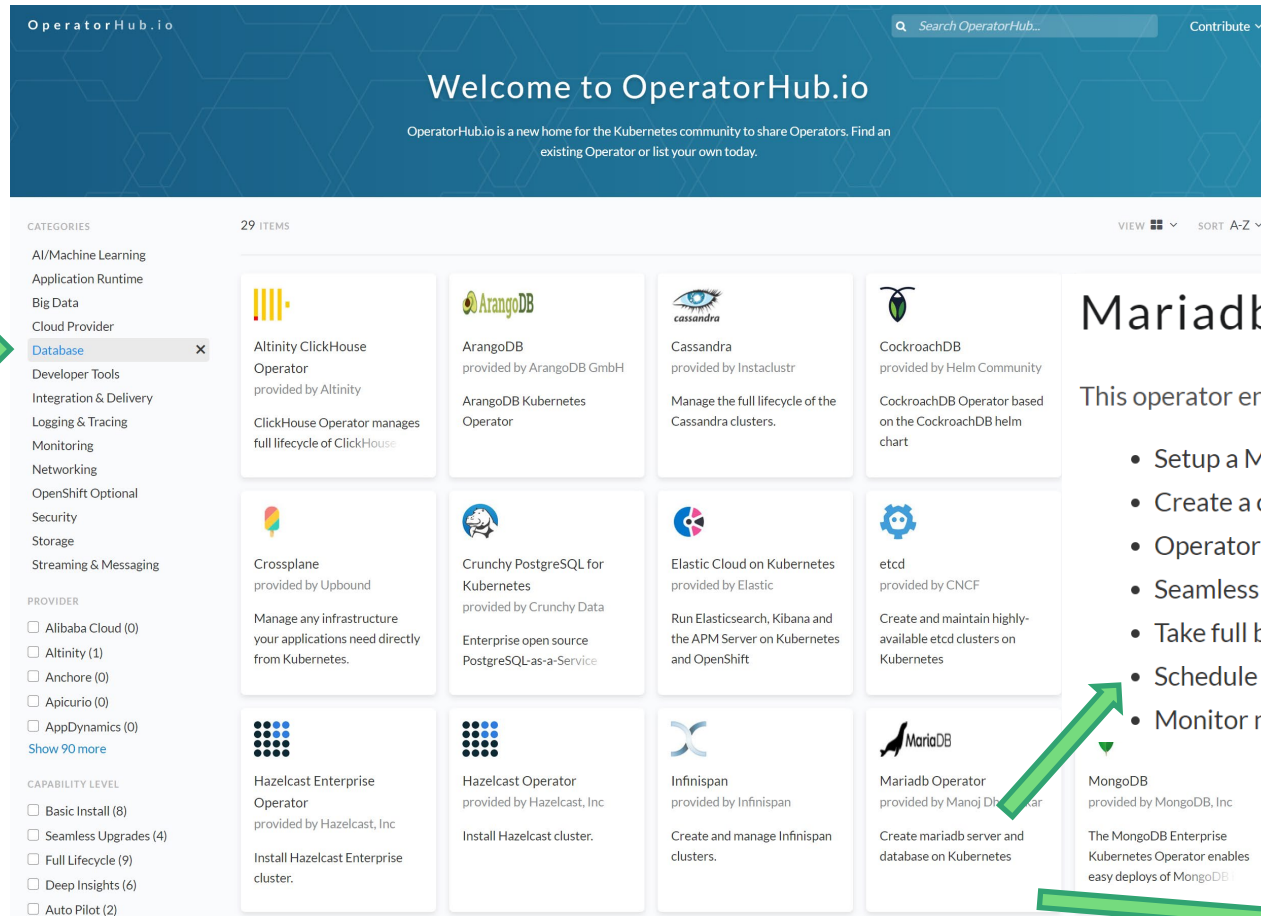
Das Operator Pattern

- Das Operator-Pattern ist angelehnt an einen menschlichen Operator (Bediener), der einen Dienst oder eine Reihe von Diensten verwaltet.
- Mit dem Operator Pattern, kann man Aufgaben automatisieren und so die Grundfunktionalität von Kubernetes erweitern ohne Kubernetes selber ändern zu müssen.
- Beispiele für Automatisierungen
 - ◇ Bereitstellen einer Anwendung bei Bedarf
 - ◇ Erstellen und Wiederherstellen von Sicherungen einer Anwendung
 - ◇ Behandlung von Upgrades (CD) des Anwendungscodes sowie der damit verbundenen Änderungen wie Datenbankschemata oder zusätzliche Konfigurationseinstellungen
 - ◇ Zur Verfügungstellung eines Dienstes für Anwendungen, die Kubernetes-APIs nicht unterstützen, z.B. damit diese Anwendungen neue Pods starten können.
 - ◇ Simulation von Fehlern in Ihrem Cluster, um dessen Ausfallsicherheit zu testen

Operator Pattern – Anwendungsfall

- Eine benutzerdefinierte Ressource namens SampleDB, die Sie im Cluster ausrollen können.
- Der Operator programmiert mittels des [Operator Frameworks](#), verpackt als Container Image.
- Der Operator der als Pods läuft.
- Verhalten
 - ◇ Wenn Sie eine neue SampleDB hinzufügen: richtet der Operator PersistentVolumeClaims ein, ein StatefulSet zum Ausführen von SampleDB und einen (Init-)Job für die Erstkonfiguration.
 - ◇ Wird die SampleDB Instanz gelöscht, erstellt der Operator einen Snapshot und stellt sicher, dass auch StatefulSet und Volumes entfernt werden.
 - ◇ Der Operator verwaltet regelmässige Datenbanksicherungen.
 - ◇ Der Operator überprüft, ob in der Datenbank eine alte Version ausgeführt wird, und erstellt Jobs für die Aktualisierung.

Operator Pattern – OperatorHub.io



OperatorHub.io

Welcome to OperatorHub.io

OperatorHub.io is a new home for the Kubernetes community to share Operators. Find an existing Operator or list your own today.

29 ITEMS

CATEGORIES

- AI/Machine Learning
- Application Runtime
- Big Data
- Cloud Provider
- Database**
- Developer Tools
- Integration & Delivery
- Logging & Tracing
- Monitoring
- Networking
- OpenShift Optional
- Security
- Storage
- Streaming & Messaging

PROVIDER

- ☐ Alibaba Cloud (0)
- ☐ Altinity (1)
- ☐ Anchore (0)
- ☐ Apicurio (0)
- ☐ AppDynamics (0)
- [Show 90 more](#)

CAPABILITY LEVEL

- ☐ Basic Install (8)
- ☐ Seamless Upgrades (4)
- ☐ Full Lifecycle (9)
- ☐ Deep Insights (6)
- ☐ Auto Pilot (2)

Operators listed:

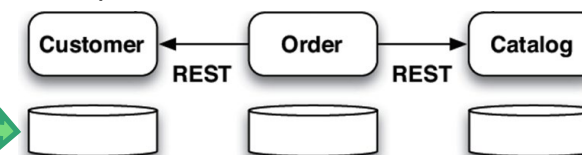
- Altinity ClickHouse Operator** provided by Altinity: ClickHouse Operator manages full lifecycle of ClickHouse
- ArangoDB** provided by ArangoDB GmbH: ArangoDB Kubernetes Operator
- Cassandra** provided by Instaclustr: Manage the full lifecycle of the Cassandra clusters.
- CockroachDB** provided by Helm Community: CockroachDB Operator based on the CockroachDB helm chart
- Crossplane** provided by Upbound: Manage any infrastructure your applications need directly from Kubernetes.
- Crunchy PostgreSQL for Kubernetes** provided by Crunchy Data: Enterprise open source PostgreSQL-as-a-Service
- Elastic Cloud on Kubernetes** provided by Elastic: Run Elasticsearch, Kibana and the APM Server on Kubernetes and OpenShift
- etcd** provided by CNCF: Create and maintain highly-available etcd clusters on Kubernetes
- Hazelcast Enterprise Operator** provided by Hazelcast, Inc: Install Hazelcast Enterprise cluster.
- Hazelcast Operator** provided by Hazelcast, Inc: Install Hazelcast cluster.
- Infinispan** provided by Infinispan: Create and manage Infinispan clusters.
- Mariadb Operator** provided by Manoj Dhanraj: Create mariadb server and database on Kubernetes
- MongoDB** provided by MongoDB, Inc: The MongoDB Enterprise Kubernetes Operator enables easy deploys of MongoDB

Mariadb Operator

This operator enables you to create MariaDB server and database easily by defining simple

- Setup a MariaDB server with configured version
- Create a custom database along with a user credential set for the custom database
- Operator uses Persistent Volume where MariaDB can write its data files
- Seamless upgrades of MariaDB is possible without losing existing data
- Take full backup of Database at user defined location
- Schedule backup at regular intervals
- Monitor metrics for mariadb node

Synchronise Microservices





Übung: Operator Pattern

- Erzeugt eine neue Custom Ressource inkl. Operator Pattern.
- Wechselt dazu, via Desktop Icon, in die "dukmaster" VM.
- Clont das Repository mit den Übungen und wechselt ins Arbeits-Verzeichnis
 - ◇ `git clone https://gitlab.com/ch-tbz-hf/Stud/v-cnt.git`
 - ◇ `cd v-cnt/2_Unterrichtsressourcen/I/custom/`
 - ◇ `kubectl apply -f resourcedefinition.yaml \`
`-f myservice-operator-rbac.yaml -f myservice-operator.yaml`
 - ◇ `kubectl apply -f myservice.yaml`
 - ◇ `kubectl get myservice`
 - ◇ `kubectl get all`



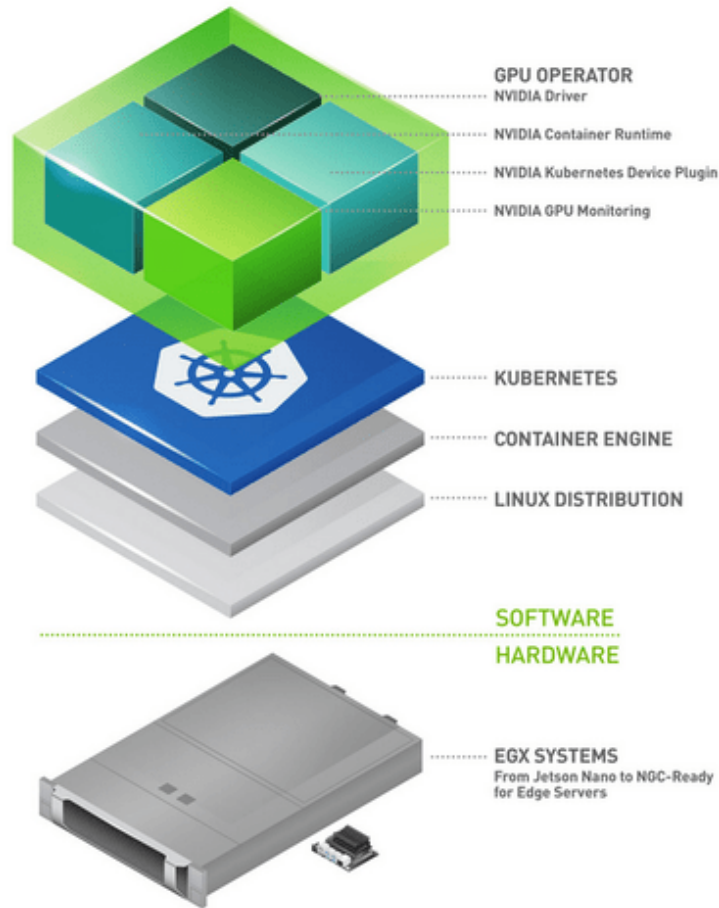
```
apiVersion: "education.tbz.ch/v1"
kind: MyService
metadata:
  name: hello-world
spec:
  image: registry.gitlab.com/mc-b/misegr/hello-world:V1.0
  replicas: 3
```

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  # name must match the spec fields below, and be in the form: <plural>.<group>
  name: myservices.education.tbz.ch
spec:
  # group name to use for REST API: /apis/<group>/<version>
  group: education.tbz.ch
  # list of versions supported by this CustomResourceDefinition
  versions:
    - name: v1
      # Each version can be enabled/disabled by Served flag.
      served: true
      # One and only one version must be marked as the storage version.
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                image:
                  type: string
                replicas:
                  type: integer
                  default: 1
                port:
                  type: integer
                  default: 80
```



- Beispiel aus dem HF-Lehrgang: [Dipl. Informatiker/in HF „Cloud-native Engineer“](#) an der TBZ.

NVIDIA GPU Operator



- GPU-Operatoren und -Plugins
 - ◇ Lösung zum Bereitstellen und Verwalten von GPUs in Kubernetes.
 - ◇ Erweiterte Funktionen wie vGPU, Multi-Instance GPU (MIG) und GPU Time-Slicing.
 - ◇ KI-Workloads auf bestimmten Knoten ausführen.
 - ◇ GPUs schnelle Kommunikationsfähigkeiten mit anderen Anwendungen/GPUs sowie mit dem Speicher-Setup.
 - ◇ GPUDirect RDMA , GPU Direct Storage und GDR Copy.



Monitoring

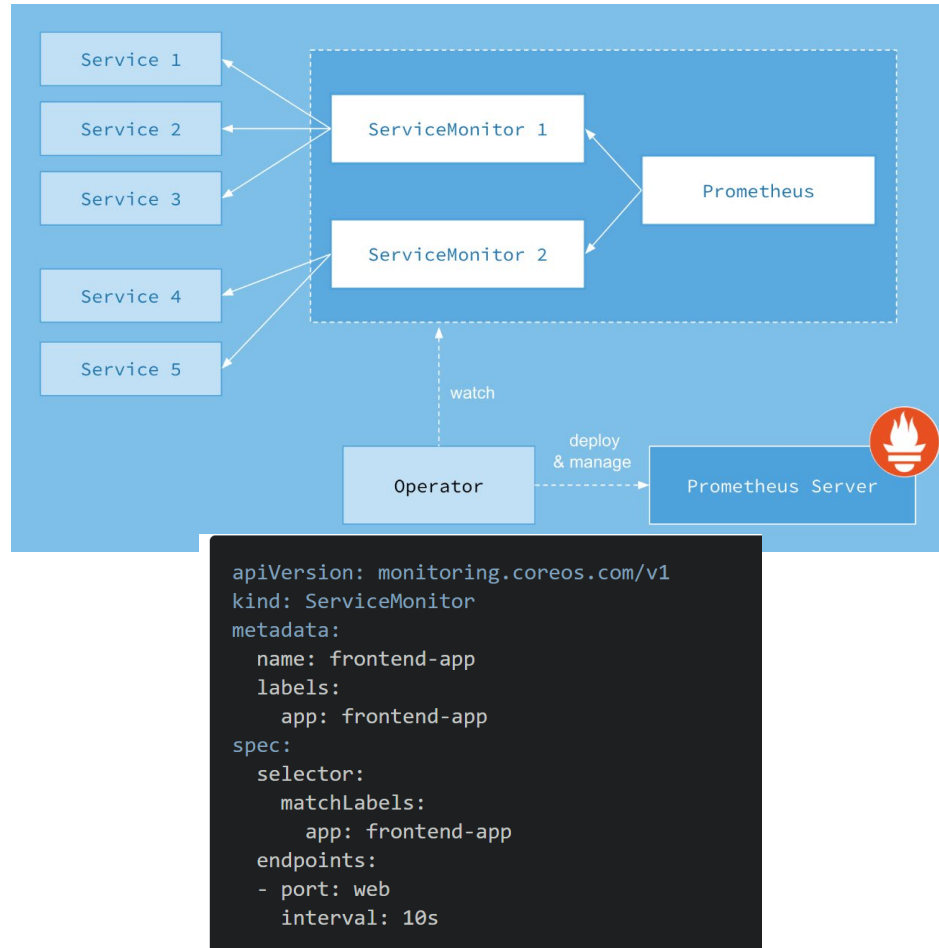
Monitoring – Vorbetrachtungen

- Manuelles Überwachen ständig sterbender, neu gestarteter sowie veränderter Services, Container und sonstiger Ressourcen im Container-Cluster stellt komplett neue Anforderungen an das Monitoring.
- Setups wie in konventionellen Sysadmin-Umgebungen auf Blech oder VMs sind hier weder praktikabel oder sinnvoll noch in irgendeiner Form realisierbar.

Monitoring Open-Source-Stack

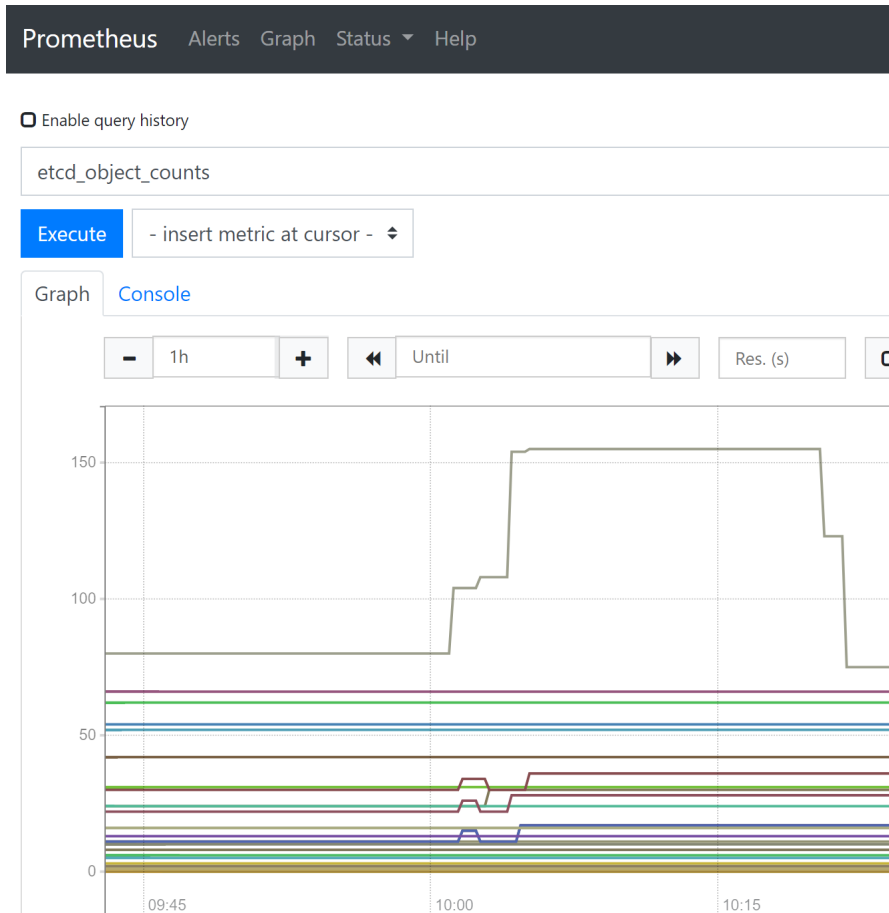
- **Prometheus** ist ein Zeitreihendatenbank- (time series database) und Überwachungstool, mit dem Metrik Endpunkte abgefragt und die von diesen Endpunkten bereitgestellten Daten abgeholt und verarbeitet werden. Daten können mit [PromQL](#) abgefragt werden.
- **Grafana** ist ein Datenvisualisierungs- und Analysetool, mit dem Sie Dashboards und Diagramme für Ihre Metrikdaten erstellen können.
- **Alertmanager** , der normalerweise zusammen mit Prometheus bereitgestellt wird, bildet die Warnschicht des Stapels. Er verarbeitet von Prometheus generierte Warnungen und dedupliziert (eliminieren von redundanten Daten), gruppiert und leitet sie z.B. via E-Mail weiter.
- **Alle drei Produkte werden mittels einem Helm Chart installiert.**
- Siehe auch: [13 Best Kubernetes Monitoring Tools: Free, Open-source & Paid](#)

Monitoring – Prometheus Funktionsweise



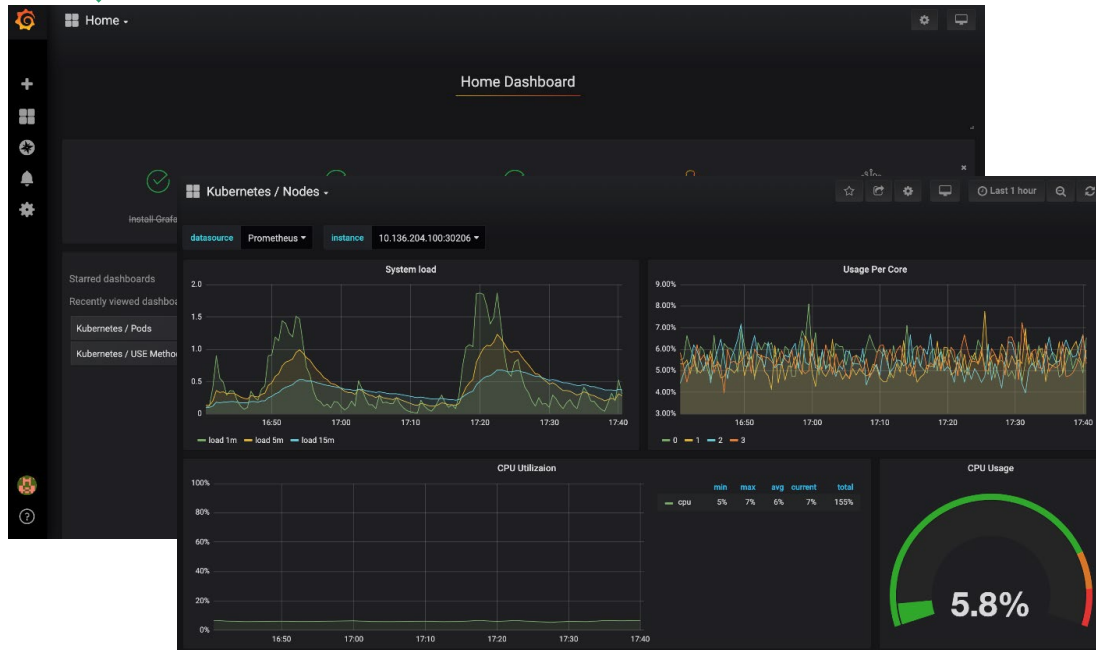
- Prometheus
 - ◇ Monitoring Lösung für K8s
 - ◇ CNCF Projekt
- Prometheus Aufgaben
 - ◇ Proaktive Überwachung
 - ◇ Sichtbarkeit und Kapazitätsplanung von Clustern
 - ◇ Alarme und Benachrichtigung auslösen
 - ◇ Metrics-Dashboards
- Prometheus Service Monitor
 - ◇ Erweiterung K8s
 - ◇ **Legt, mittels Selektoren, fest was Überwacht wird.**

Monitoring – Prometheus User Interface



- **Prometheus** ist ein Zeitreihendatenbank- (time series database) und Überwachungstool, mit dem Metrik Endpunkte abgefragt und die von diesen Endpunkten bereitgestellten Daten abgeholt und verarbeitet werden. Daten können mit [PromQL](#) abgefragt werden.
- Siehe auch:
 - ◇ [Prometheus Query Language](#)
 - ◇ An [Introduction](#) to PromQL

Monitoring – Grafana



- **Grafana** ist ein Datenvisualisierungs- und Analysetool, mit dem Sie Dashboards und Diagramme für Ihre Metrikdaten erstellen können.
- Auf der linken Seite stehen eine Vielzahl von vorbereiteten Dashboards zur Verfügung.
- Diese Dashboards werden von [kubernetes-mixin](#) einem Open-Source-Projekt generiert, welches einen standardisierten Satz von Grafana-Dashboards und Prometheus-Warnungen zur Clusterüberwachung erstellt.

Monitoring – Alertmanager

Alertmanager Alerts Silences Status Help

Filter Group Receiver: All ☐ Silence

Custom matcher, e.g. `env="production"`

+ Expand all groups

+ Not grouped 1 alert

+ Not grouped 3 alerts

- job="kube-etcd" + 2 alerts

09:24:33, 2020-02-28 (UTC) + Info Source Silence

alertname="etcdInsufficientMembers" + prometheus="monitoring/prometheus-operator-prometheus" +

09:31:45, 2020-02-28 (UTC) + Info Source Silence

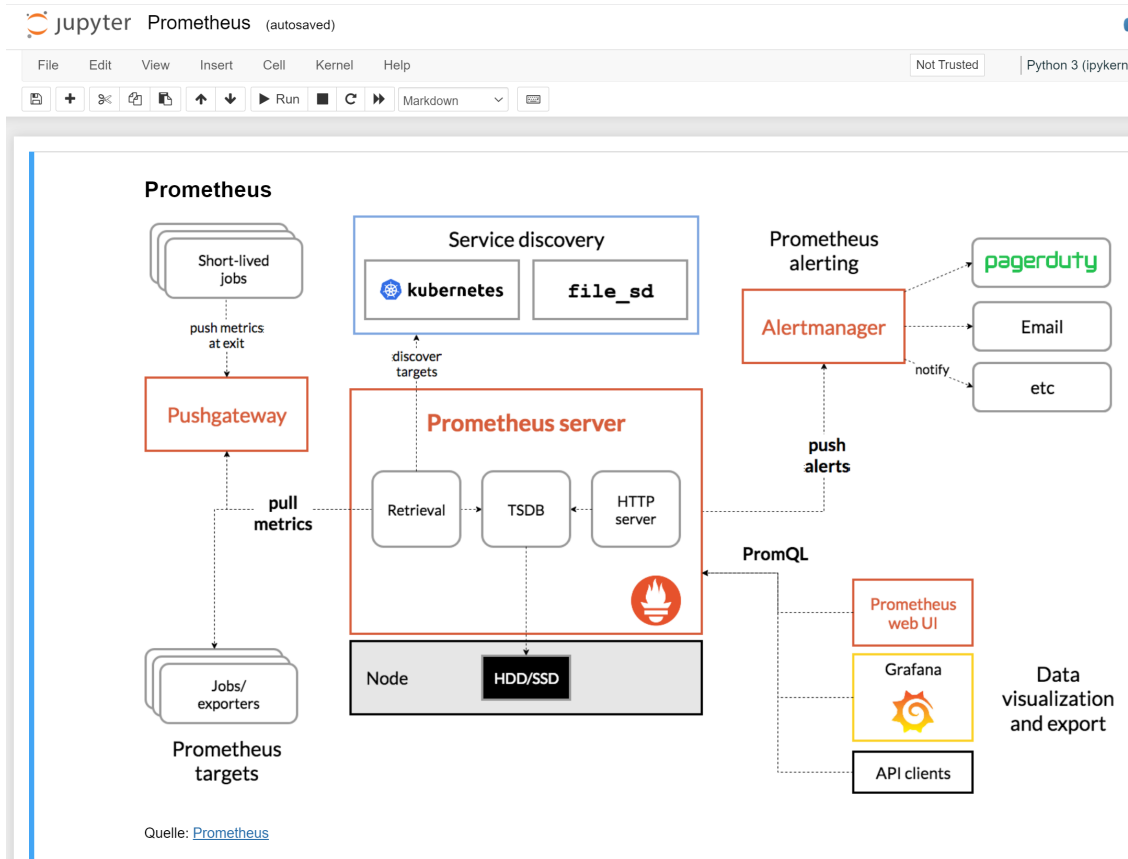
alertname="TargetDown" + namespace="kube-system" + prometheus="monitoring/prometheus-operator-prometheus" +

service="prometheus-operator-kube-etcd" + severity="warning" +

- **Alertmanager**, bildet die Warnschicht des Stapels. Er verarbeitet von Prometheus generierte Warnungen.
- Offizielle Alertmanager Dokumentation
 - ◇ <https://prometheus.io/docs/alerting/alertmanager/>
- Notifikation Beispiel
 - ◇ https://prometheus.io/docs/alerting/notification_examples/



Übung: Monitoring





Debugging/Troubleshooting

Debugging/Troubleshooting (1)

- Troubleshooting und Debugging allein würde ein weiteres komplettes Buch füllen.
- Zudem wäre selbst ein umfangreicher Debugging-Leitfaden immer nur so aktuell wie die vorliegende K8s-Version.
- Daher ist einer der flexibelsten Ansätze, die relativ gut dokumentierten Debugging und Troubleshooting-Seiten des K8s Projekts aufzurufen:
 - ◇ **Allgemeine Troubleshooting- und Debugging-FAQs und generelle Verweise**
 - <https://github.com/kubernetes/kubernetes/wiki/Debugging-FAQ>
 - <http://kubernetes.io/docs/troubleshooting/>
 - <https://kubernetes.io/docs/tasks/debug-application-cluster/>
 - ◇ **Cluster-Troubleshooting**
 - <http://kubernetes.io/docs/admin/cluster-troubleshooting/>
 - ◇ **Pods in Produktivumgebungen debuggen**
 - **How to Troubleshoot Applications on Kubernetes**
 - <https://blog.alexellis.io/troubleshooting-on-kubernetes/>
 - <http://kubernetes.io/docs/user-guide/production-pods/>
 - <https://xenitab.github.io/blog/2022/04/12/ephemeral-container-security/>

Debugging/Troubleshooting (2)

- **Pods und ReplicaController debuggen**
 - ◇ <http://kubernetes.io/docs/user-guide/debugging-pods-and-replication-controllers/>
- **Services und Network debuggen**
 - ◇ <https://kubernetes.io/docs/tasks/debug-application-cluster/local-debugging/>
 - ◇ <https://www.digitalocean.com/community/tutorials/how-to-inspect-kubernetes-networking>
 - ◇ <https://kinvolk.io/blog/2020/04/inside-kinvolk-labs-investigating-kubernetes-performance-issues-with-bpf/>
- **Applikationsspezifisches Debugging in den Pods**
 - ◇ <https://kubernetes.io/docs/tasks/debug-application-cluster/debug-application-introspection/>
 - ◇ 5 [tips for troubleshooting apps](#) on Kubernetes
- **Audits**
 - ◇ <https://kubernetes.io/docs/tasks/debug-application-cluster/audit/>
- **Zustand der Ressourcen und des Cluster als Dump (inkl. Log pro Pod)**
 - ◇ `kubectl cluster-info dump --output-directory=k8sdump`

Debugging/Troubleshooting (3)

- Artikel aus KubeWeekly
 - ◇ Speed up administration of Kubernetes clusters with [k9s](#).
 - ◇ DNS issues in Kubernetes. Public [postmortem](#) #1
 - ◇ [Mistakes made and lessons learned](#) with Kubernetes and GitOps
 - ◇ [BEST PRACTICES](#) FOR MICROSERVICES ON KUBERNETES
 - ◇ Managing Kubernetes: [7 things](#) you should understand

Kubernetes Security (Empfehlungen) (1)

- Docker CLI entfernen oder anderes Container Runtime statt Docker verwenden.
- Der Netzwerktrafik mittels [Network Policy Provider](#) eingrenzen
- Ressourcen pro Namespace begrenzen
- Pods verwenden Ausschliesslich PersistentVolumeClaims, nie hostPath!
- [Security Hardening](#) Kubernetes
- Verwendet Policy Management Tools, wie z.B. <https://kyverno.io/>
- NSA and CISA Kubernetes [Security Guidance](#)
- <https://falco.org/> – Falco , das Cloud-native Runtime-Sicherheitsprojekt, ist die De-facto- Engine von Kubernetes zur Erkennung von Bedrohungen
- <https://kubescape.io/> – Eine Open-Source-Kubernetes-Sicherheitsplattform für Ihre IDE, CI/CD-Pipelines und Cluster

Kubernetes Security (Empfehlungen) (2)

- Lesen
 - ◇ <https://12factor.net/>
 - ◇ [Kubernetes Security Audit](#)
 - ◇ [Tools and Methods for Auditing Kubernetes RBAC Policies](#)
 - ◇ [12 Kubernetes configuration](#) best practices
 - ◇ Kubernetes deployments: [6 security best practices](#)
 - ◇ Gartner: [6 Best Practices](#) for Containers, Kubernetes
 - ◇ [Unveil hidden malicious processes](#) with Falco in cloud-native environments

Mehr Kubernetes Day 2

Kubernetes 2-Day Operations mit CNCF («K8SD2O»)

Erkunden Sie im Kurs das CNCF-Ökosystem rund um Kubernetes. Lernen Sie in zwei Tagen, wie Sie durch den Einsatz ausgesuchter CNCF-Projekte Ihre Kubernetes-Infrastruktur und Ihre Microservices sicherer, effizienter und nachhaltiger betreiben können.

- <http://digicomp.ch/d/k8sd2o>



Reflexion

- **Helm** hilft Ihnen bei der Verwaltung von Kubernetes-Anwendungen – mit Helm Charts können Sie selbst die komplexeste Kubernetes-Anwendung definieren, installieren und aktualisieren.
- Das **Operator Pattern**, erlaubt es eigene **Automatisierungen** in Kubernetes zu implementieren ohne Kubernetes selber ändern zu müssen.
- **Monitoring** unterscheidet sich bei Kubernetes, u.a. wegen der kurz lebigkeit, vom klassischen Ansatz. Prometheus und andere Tools füllen langsam die Lücke.
- **Troubleshooting und Debugging** allein würde ein weiteres komplettes Buch füllen, besser die Debugging und Troubleshooting-Seiten des K8s Projekts aufzurufen.
- Für alle weiteren Anforderungen, wie **Logging, Service Mesh, Workflow, Serverless** etc., entstehen fast täglich neue Produkte rund um Kubernetes.

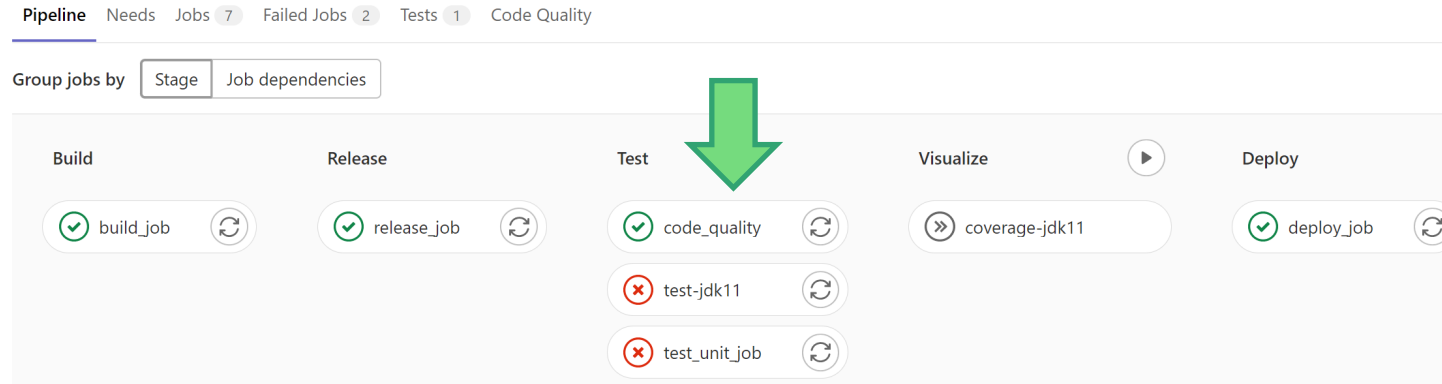
Lernzielkontrolle

- Sie haben einen Überblick über Erweiterte Funktionen im Kubernetes Cluster.

Continuous Integration und Delivery – Definition

- **Continuous Integration (CI)** (auch fortlaufende oder permanente Integration) ist ein Begriff aus der Software-Entwicklung, der den Prozess des fortlaufenden Zusammenfügens von Komponenten zu einer Anwendung beschreibt.
 - ◇ Lösbar mittels Dockerfile, besser CI/CD Systeme wie Jenkins, GitLab etc.
- Continuous Delivery ist ein Schritt über Continuous Integration hinaus. Ihre Anwendung wird nicht nur jedes Mal erstellt und getestet, wenn eine Codeänderung in die Codebasis übertragen wird, sondern die Anwendung wird auch kontinuierlich bereitgestellt. Bei Continuous Delivery lösen Sie die **Bereitstellungen** jedoch **manuell** aus.
 - ◇ 1. Teil: docker build (mit Tag!) und docker push
- Continuous Deployment ist ein weiterer Schritt über Continuous Integration hinaus, ähnlich wie Continuous Delivery. Der Unterschied besteht darin, dass Sie Ihre Anwendung nicht manuell bereitstellen, sondern automatisch bereitstellen. Menschliches Eingreifen ist nicht erforderlich
 - ◇ 2. Teil: Deployment in Kubernetes

Beispiel: GitLab



Beispiel aus CDI (DevOps Engineering Practices & Tools) Kurs

<https://gitlab.com/ch-mc-b/cdi/apps/web-app-full>

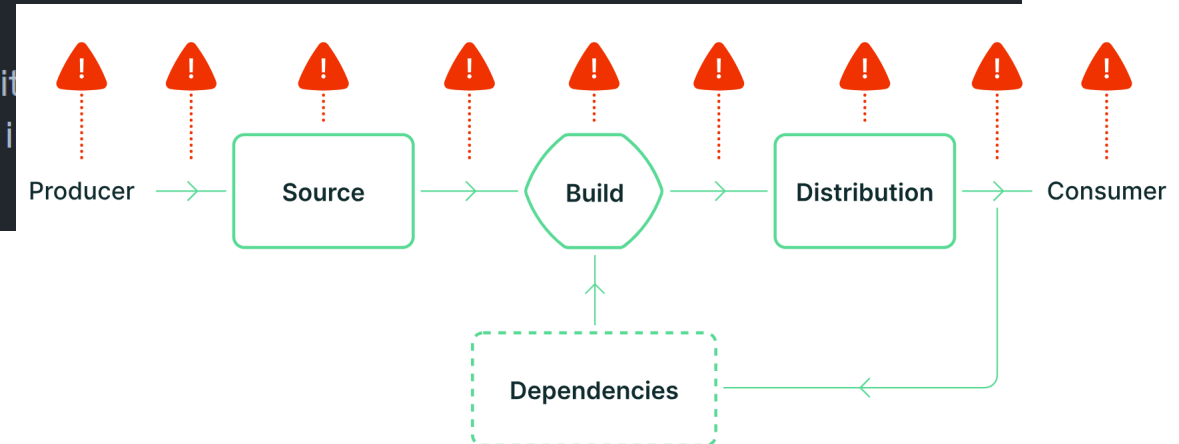
Static Application Security Testing (SAST)

- Ihr Code enthält ein potenziell gefährliches Attribut in einer Klasse (z.B. Password) oder unsicheren Code, der zu einer unbeabsichtigten Codeausführung führen kann.
- Ihre Anwendung ist anfällig für Cross-Site-Scripting-Angriffe (XSS), die für den unbefugten Zugriff auf Sitzungsdaten ausgenutzt werden können.

Beispiel: GitHub

Enhance build security and reach SLSA Level 3 with GitHub Artifact Attestations

Learn how GitHub Artifact Attestations can enhance your build security and reach SLSA Level 3. This post breaks down the basics of SLSA, explains the importance of attestations, and provides a step-by-step guide to securing your build process.



Continuous Integration mit Docker

 **Dockerfile**  208 Bytes

```

1 FROM golang:1.16-alpine AS build
2
3 WORKDIR /src/
4 COPY *.go go.* /src/
5 RUN CGO_ENABLED=0 go build -o /bin/birdpedia
6
7 FROM scratch
8 COPY --from=build /bin/birdpedia /bin/birdpedia
9 COPY assets/index.html /assets/

```

```

1 FROM mcr.microsoft.com/dotnet/core/aspnet:3.1-buster-slim AS base
2 WORKDIR /app
3 EXPOSE 80
4 FROM mcr.microsoft.com/dotnet/core/sdk:3.1-buster AS build
5 WORKDIR /src
6 COPY ["DockerDemo.csproj", ""]
7 RUN dotnet restore "./DockerDemo.csproj"
8 COPY . .
9 WORKDIR "/src/"
10 RUN dotnet build "DockerDemo.csproj" -c Release -o /app/build
11 FROM build AS publish
12 RUN dotnet publish "DockerDemo.csproj" -c Release -o /app/publish
13 FROM base AS final
14 WORKDIR /app
15 COPY --from=publish /app/publish .
16 ENTRYPOINT ["dotnet", "DockerDemo.dll"]

```

- Im Dockerfile sind mehrere **FROM** Einträge möglich, sogenannte multi-stage builds.
- Das kann benutzt werden um:
 - ◇ Ein Base Image mit Compiler, Build Tools etc. zu starten und die Sourcen zu bauen
 - ◇ Von einem neuen Base Image abzuleiten, z.B. mit dem Runtime der entsprechenden Sprache.
 - ◇ Das gebildete Programm vom ersten Container Image in das zweite Container Image zu kopieren.
 - <https://gitlab.com/mc-b/birdpedia/-/tree/master>
 - <https://codefresh.io/docker-tutorial/docker-images-net-core/>
 - **Hinweis:** Visual Studio Enterprise verwendet diese Variante auch.

Alternative HELM?: Kustomize

base: kustomization + resources

kustomization.yaml

```
commonLabels:
  app: myWord
resources:
- deployment.yaml
- service.yaml
configMapGenerator:
- name: wordpress-map
  files:
  - env.startup.txt
```

deployment.yaml

```
apiVersion: v1
kind: Deployment
metadata:
  name: wordpress
  labels:
    app: wordpress
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress
  template: ...
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress
spec:
  ports:
  - port: 389
  selector:
    app: wordpress
```

- Mit kustomize kann man YAML-Dateien für mehrere Zwecke anpassen.
- Die ursprüngliche YAML bleibt dabei unverändert.
- `kubectl apply -k .` – integriert in Kubectl
- Dokumentation: <https://github.com/kubernetes-sigs/kustomize>
- Beispiele: <https://github.com/kubernetes-sigs/kustomize/tree/master/examples>
- Vergleich: <https://povilasv.me/helm-kustomze-better-together/>